

PageRank Algorithm Analysis

Final Report

Authors:

Lucio Baiocchi (ID: 360244)

Leonardo Passafiume (ID: 358616)

Computational Linear Algebra for Large Scale Problems
Politecnico di Torino
Academic Year 2025/2026

Contents

1	Numerical Experiments and Exercises	2
1.1	Exercise 1: Manipulating PageRank	2
1.1.1	Problem Statement	2
1.1.2	Implementation	2
1.1.3	Results and Analysis	3
1.2	Exercise 4: Dangling Nodes and Substochastic Matrices	4
1.2.1	Problem Statement	4
1.2.2	Implementation	4
1.2.3	Results and Interpretation	5
1.3	Exercise 13: Disconnected Subwebs	5
1.3.1	Problem Statement	5
1.3.2	Implementation	6
1.3.3	Results and Analysis	7
2	PageRank Algorithm Implementation	8
2.1	Data Structure: The Graph	8
2.2	The Power Method	9
2.3	Analysis of Results	11
2.3.1	Case 1: Simple Network	11
2.3.2	Case 2: Disconnected Components	11
2.3.3	Case 3: "Hollins" Dataset	12
2.4	Conclusions on Implementation	13
2.5	Experimental Results	13
2.5.1	Case 1: Standard Verification (Figure 2.1)	14
2.5.2	Case 2: Rank Manipulation Strategy	14
2.5.3	Case 3: Real-World Dataset (Hollins.dat)	14

Chapter 1

Numerical Experiments and Exercises

In this chapter, we solve specific exercises related to the manipulation of web structures and the analysis of dangling nodes (from the following [paper](#)). The calculations were performed using the Python `numpy` library.

1.1 Exercise 1: Manipulating PageRank

1.1.1 Problem Statement

We consider the scenario where the owners of Page 3 are unsatisfied with their ranking. In the original web graph (Figure 2.1 from [paper](#)), Page 1 has a higher score than Page 3. To attempt to boost Page 3's score, a new page (Page 5) is created. Page 5 links to Page 3, and Page 3 includes a link to Page 5. We must determine if this strategy successfully raises Page 3's rank above Page 1.

1.1.2 Implementation

We define the original link matrix A and the modified matrix A_{mod} . In A_{mod} , the dimensions are increased to 5×5 .

- Page 5 (column 5) links only to Page 3.
- Page 3 (column 3) now splits its votes between Page 1 and Page 5.

The Python implementation for the matrix definition is as follows:

```
1 import numpy as np
2
3 # Original Matrix (4 nodes)
4 A = np.array([
5     [0, 0, 1, 1/2],
6     [1/3, 0, 0, 0],
7     [1/3, 1/2, 0, 1/2],
8     [1/3, 1/2, 0, 0]
```

```

9  ])
10
11  # Modified Matrix (5 nodes)
12  # Col 2 (Page 3) now links to Page 1 (Row 0) and Page 5 (Row
13    4)
14  # Col 4 (Page 5) links to Page 3 (Row 2)
15  A_mod = np.array([
16      [0, 0, 1/2, 1/2, 0],
17      [1/3, 0, 0, 0, 0],
18      [1/3, 1/2, 0, 1/2, 1],
19      [1/3, 1/2, 0, 0, 0],
20      [0, 0, 1/2, 0, 0]
  ])

```

Listing 1.1: Definition of Original and Modified Link Matrices

Both matrices were verified to be column-stochastic (columns sum to 1). We then computed the eigenvectors for the eigenvalue $\lambda = 1$ and normalized them using the L_1 norm (sum of components equals 1).

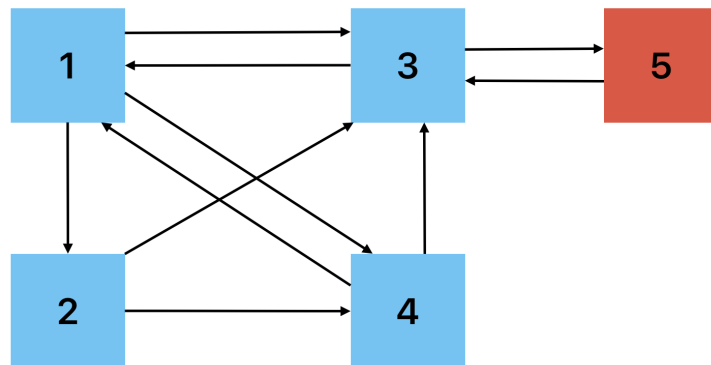


Figure 1.1: Image of the Matrix taken in consideration for the exercise 1.

1.1.3 Results and Analysis

The computed Importance Vectors (PageRank) for both scenarios are compared below.

Page	Original Score (x_k)	Modified Score (x'_k)
1	0.387	0.245
2	0.129	0.082
3	0.290	0.367
4	0.194	0.122
5	N/A	0.184

Table 1.1: Comparison of PageRank scores before and after adding Page 5.

Conclusion: The strategy was successful. In the original configuration, Page 1 was the highest-ranked ($0.387 > 0.290$). In the modified web, Page 3's score rose to 0.367, while Page 1's score dropped to 0.245. By creating a feedback loop with Page 5, Page 3 successfully retained more "probability mass" within its local subgraph, effectively boosting its importance at the expense of Page 1.

1.2 Exercise 4: Dangling Nodes and Substochastic Matrices

1.2.1 Problem Statement

Starting from the original web (Figure 2.1 in paper), we modify the graph by removing the link from Page 3 to Page 1. Since Page 3 previously linked only to Page 1, removing this link makes Page 3 a *dangling node* (a page with no outgoing links). We aim to:

1. Set up the corresponding substochastic matrix B .
2. Find the largest positive eigenvalue (Perron root).
3. Compute the corresponding non-negative eigenvector and analyze the ranking.

1.2.2 Implementation

The matrix B is defined such that the column corresponding to Page 3 contains only zeros. Consequently, the matrix is no longer strictly stochastic (the sum of column 3 is 0), but *substochastic*.

```

1 # Substochastic Matrix B (Page 3 has no outgoing links)
2 B = np.array([
3     [0, 0, 0, 1/2],
4     [1/3, 0, 0, 0],
5     [1/3, 0, 0, 1/2],
6     [1/3, 1, 0, 0]
7 ])
8
9 def get_perron(matrix):
10     eigvals, eigvecs = np.linalg.eig(matrix)
11     # Find index of the eigenvalue with largest magnitude
12     idx = np.argmax(np.abs(eigvals))
13
14     perron_val = eigvals[idx].real
15     perron_vec = eigvecs[:, idx].real
16
17     # Ensure non-negative components
18     if perron_vec[0] < 0:
19         perron_vec = -perron_vec
20

```

```

21     # Normalize (L1 Standardizer)
22     return perron_val, perron_vec / perron_vec.sum()
23
24 val, vec = get_perron(B)

```

Listing 1.2: Substochastic Matrix and Perron Eigenvector Computation

1.2.3 Results and Interpretation

The computational results yield:

- **Perron Eigenvalue** (λ_{max}): ≈ 0.650
- **Perron Eigenvector** (v): $[0.231, 0.118, 0.350, 0.301]^T$

The resulting ranking is:

Page 3 $\succ$ Page 4 $\succ$ Page 1 $\succ$ Page 2

Discussion: The largest eigenvalue is $\lambda \approx 0.65 < 1$. This confirms that the system loses probability mass over time; a random surfer will eventually enter Page 3 and get "stuck" (or disappear, mathematically speaking).

Despite being a dead end, Page 3 achieves the highest rank (0.350). This result is reasonable within the context of the random surfer model on a substochastic matrix: since all paths eventually lead toward the sink (Page 3) and there is no way out, the surfer spends the last moments of their session at Page 3. Therefore, the node effectively acts as an accumulator of importance before the surfer terminates the session.

1.3 Exercise 13: Disconnected Subwebs

1.3.1 Problem Statement

In this exercise, we construct a web graph consisting of two distinct, disconnected components (subwebs) and determine the global ranking using the standard PageRank formula (referred to as Formula 3.1 in the paper).

The web consists of $n = 7$ pages divided into two isolated clusters:

- **Subweb A:** Nodes 1, 2, 3, and 4.
- **Subweb B:** Nodes 5, 6, and 7.

There are no links connecting Subweb A to Subweb B or vice versa. We aim to observe how the introduction of the teleportation parameter (damping factor α) allows for a unique ranking vector despite the reducibility of the adjacency matrix.

1.3.2 Implementation

The graph structure is defined using our custom Python implementation where nodes and edges are explicitly added. The connectivity is described as follows:

- **Node 1** links to 2, 3, 4.
- **Node 2** links to 3, 4.
- **Node 3** links to 1.
- **Node 4** links to 1, 3.
- **Node 5** links to 7.
- **Node 6** links to 5.
- **Node 7** links to 5, 6.

The Python code used to generate the graph and compute the rank is provided below:

```

1 from functions.graph import *
2 from functions.functions import *
3
4 # Create the nodes
5 nodes = [Node(number=i) for i in range(1, 8)]
6 n1, n2, n3, n4, n5, n6, n7 = nodes
7
8 # Linking edges (defining Out-links)
9 # Subweb A (Nodes 1-4)
10 n1.addOut([2, 3, 4])
11 n2.addOut([3, 4])
12 n3.addOut([1])
13 n4.addOut([1, 3])
14
15 # Subweb B (Nodes 5-7)
16 n5.addOut([7])
17 n6.addOut([5])
18 n7.addOut([5, 6])
19
20 # Initialize Graph and Compute Rank
21 graph = Graph(nodes)
22 graph.print()
23 # graph.plot() # Visualization
24 testRank(graph)

```

Listing 1.3: Construction of Disconnected Subwebs

Mathematically, this results in a block-diagonal column-stochastic matrix S . Since the graph has no dangling nodes, the Google Matrix M is computed as:

$$M = (1 - m)Ax + mS$$

where m is the damping factor (typically 0.15).

1.3.3 Results and Analysis

The algorithm converged after 34 iterations (difference $< 10^{-8}$). The resulting global ranking is presented in Table 1.2.

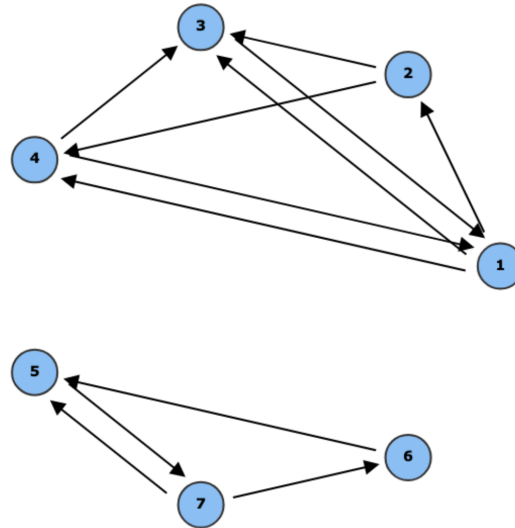


Figure 1.2: Visual representation of the two disconnected subwebs.

Rank	Score	Node ID	Subweb
1	0.2104	1	A
2	0.1703	5	B
3	0.1662	7	B
4	0.1645	3	A
5	0.1155	4	A
6	0.0921	6	B
7	0.0810	2	A

Table 1.2: PageRank results for the disconnected graph (Exercise 13).

Discussion: Although the two subwebs are disconnected, the random surfer model (via the teleportation vector) successfully ranks all pages on a single scale.

We observe that the total probability mass is roughly distributed according to the size of the subwebs. Subweb A (4 nodes) holds approximately ≈ 0.57 of the total score, while Subweb B (3 nodes) holds ≈ 0.43 . This occurs because the random surfer teleports uniformly to any node in the network; thus, a larger component acts as a larger "trap" for the surfer, accumulating more total importance over time. Inside Subweb A, Node 1 is dominant due to receiving links from 3 and 4, while in Subweb B, Node 5 takes the lead.

Chapter 2

PageRank Algorithm Implementation

This chapter presents the computational implementation of the theoretically described method. The objective is to calculate the PageRank vector x that satisfies the eigenvalue equation for the Google Matrix M . For large-scale networks, the explicit construction of the dense matrix M is computationally prohibitive; therefore, iterative methods are employed by exploiting the sparsity of the link matrix A .

2.1 Data Structure: The Graph

To represent the web network, a data structure based on two main classes was created: **Node** and **Graph**.

The **Node** class represents a single web page and maintains the list of outgoing nodes (*out-links*) and incoming nodes (*in-links*). The **Graph** class manages the set of nodes and handles the conversion of the graph structure into a matrix representation suitable for algebraic calculation.

A fundamental step is the generation of the sparse adjacency matrix. Given a graph with N nodes, the `get_matrix_data` function constructs the matrix A (column-normalized) and identifies the so-called *dangling nodes* (nodes with no outgoing links).

```
1 def get_matrix_data(self):
2     nodes = self.nodes
3     N = len(nodes)
4     # ... List initialization for CSR matrix ...
5     h = np.zeros((N, 1)) # Dangling vector
6
7     for i, node in enumerate(nodes):
8         out_neighbors = node.outGoing
9         out_degree = len(out_neighbors)
10
11         if out_degree == 0:
12             h[i, 0] = 1.0 # Dangling node identified
13         else:
```

```

14         weight = 1.0 / out_degree
15         for target_id in out_neighbors:
16             # ... Weighted link construction ...
17             data.append(weight)
18
19     A = csr_matrix((data, (rows, cols)), shape=(N, N))
20     return A, h, self.index_to_id

```

Listing 2.1: Extraction of the sparse matrix and handling of dangling nodes

2.2 The Power Method

The core of the work is the `PowerMethod` function, which solves the principal eigenvector problem. As discussed in the theory, we seek the stationary eigenvector x such that $Mx = x$, where $M = (1 - m)A + mS$.

However, instead of operating on the dense matrix M , the implementation exploits the decomposition of the formula to maintain a complexity of $O(N)$ (where N is the number of links) rather than $O(N^2)$. The implemented iterative equation is:

$$x_{k+1} = (1 - m) (Ax_k + (h^T x_k) \cdot \mathbf{s}) + m\mathbf{s} \quad (2.1)$$

Where:

- A is the sparse link matrix;
- m is the damping factor (typically 0.15);
- $\mathbf{s}$ is the uniform personalization vector ($1/N$);
- The term $(h^T x_k)$ manages the redistribution of probability mass lost in the *dangling nodes*.

```

1 def PowerMethod(A, N, m=0.15, h=None, relTol=1e-8, maxiter
   =500):
2     # Uniform vector initialization
3     s = np.full((N, 1), 1.0 / N)
4     xk = s.copy()
5     alpha = 1.0 - m
6
7     for k in range(maxiter):
8         xk_prev = xk.copy()
9
10        # Dangling node handling (lost mass)
11        dangling_sum = h.T @ xk_prev
12        dangling_part = dangling_sum * s
13
14        # Optimized iterative calculation

```

```
15         xk_new = alpha * (A @ xk_prev + dangling_part) + (m *  
16             s)  
17         # L1 Normalization  
18         xk_new = xk_new / np.linalg.norm(xk_new, ord=1)  
19  
20         # Convergence check  
21         diff = np.linalg.norm(xk_new - xk_prev, ord=1)  
22         if diff < relTol:  
23             return xk_new  
24         xk = xk_new  
25     return xk
```

Listing 2.2: Implementation of the Power Method

Convergence is verified by calculating the L_1 norm of the difference between two successive iterations: $\|x_{k+1} - x_k\|_1 < \epsilon$.

2.3 Analysis of Results

The algorithm was tested on three distinct scenarios to verify its correctness regarding the theory and its robustness on real data.

2.3.1 Case 1: Simple Network

The first test reproduces the 4-node network described in *The \$25,000,000,000 Eigenvector* (see Figure 2.1). The network presents a strongly connected structure. The algorithm converges rapidly and assigns, as predicted theoretically, the highest importance to node 1, despite node 3 having more total incoming links. This demonstrates PageRank's ability to weigh the "quality" of links (the link from node 3 to node 1 is valuable because node 3 is exclusive).

Visualizzazione grafo

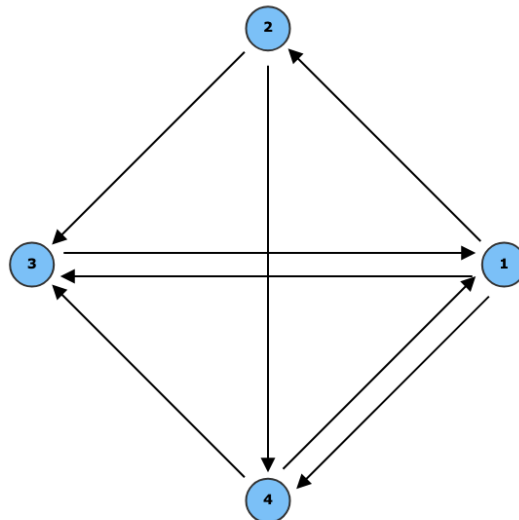


Figure 2.1: Visual representation of the simple 4-node network (Case 1).

2.3.2 Case 2: Disconnected Components

The second test introduces a node 5 disconnected from the rest of the main network, creating two isolated subwebs (Figure 2.2). Without the damping factor m (teleportation), the matrix A would be reducible, and the eigenspace for eigenvalue 1 would have a dimension greater than 1, making the ranking ambiguous. The implementation with $m = 0.15$ guarantees that the matrix M is positive and primitive, ensuring the uniqueness of the Perron eigenvector (Perron-Frobenius Theorem). The result shows that the ranking is correctly calculated across all components.

Visualizzazione grafo

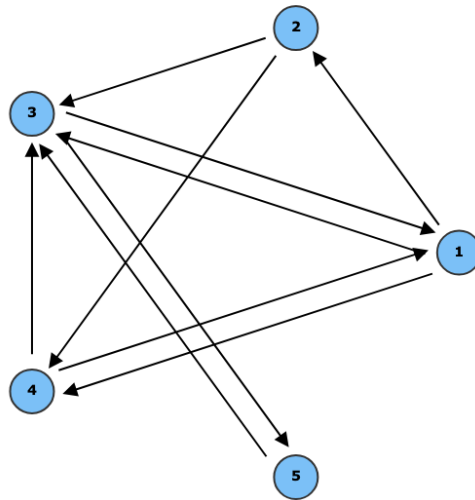


Figure 2.2: Network with disconnected components (Case 2).

2.3.3 Case 3: "Hollins" Dataset

Finally, the algorithm was applied to a real dataset (`hollins.dat`) containing a larger number of nodes. Figure 2.3 illustrates the complexity of the link structure. The results show the top 10 URLs by importance. It is interesting to note how the use of sparse matrices (`scipy.sparse.csr_matrix`) is essential at this stage: the required memory scales linearly with the number of links, allowing analysis even on graphs that could not be managed with standard dense matrices.



Figure 2.3: Visualization of the Hollins dataset graph (6012 nodes).

2.4 Conclusions on Implementation

The iterative approach proved effective. The computational complexity of each iteration is dominated by the sparse matrix-vector product, resulting in high efficiency. The explicit management of the dangling vector h and the teleportation factor m allows for solving convergence issues (sink nodes and cycles) typical of a simple eigenvector analysis on pure adjacency matrices.

2.5 Experimental Results

The implementation was tested on three distinct scenarios to verify correctness against theoretical expectations and to assess performance on real-world data.

2.5.1 Case 1: Standard Verification (Figure 2.1)

The first test reproduces the 4-node network described in the reference literature. This network serves as a baseline to verify the correctness of the `PowerMethod` implementation.

- **Convergence:** Achieved in 24 iterations (Difference $< 10^{-8}$).
- **Damping Factor (m):** 0.15

The calculated scores perfectly match the theoretical values provided in the literature ($x_1 \approx 0.368$), confirming the accuracy of the algorithm.

Rank	Score	Node ID	Description
1	0.3682	1	Linked by Node 3 (Exclusive link)
2	0.2880	3	Links to Node 1 only
3	0.2021	4	Links to 1 and 3
4	0.1418	2	Receives link from 1

Table 2.1: PageRank results for the basic 4-node web.

2.5.2 Case 2: Rank Manipulation Strategy

In this scenario, a fifth page (Node 5) was added to the network to create a feedback loop with Node 3 ($3 \leftrightarrow 5$). This corresponds to the strategy of artificially boosting a page's rank.

- **Convergence:** Achieved in 38 iterations.
- **Result:** Node 3 successfully overtook Node 1 in the ranking.

Rank	Score	Node ID	Change from Case 1
1	0.3489	3	Moved up (Prev. Rank 2)
2	0.2371	1	Moved down (Prev. Rank 1)
3	0.1783	5	New Node (Booster)
4	0.1385	4	Decreased score
5	0.0972	2	Decreased score

Table 2.2: Results of the boosting strategy. Node 3 gains the top rank.

2.5.3 Case 3: Real-World Dataset (Hollins.dat)

The algorithm was applied to the `hollins.dat` dataset to test scalability.

- **Total Nodes:** 6,012
- **Dangling Nodes:** 3,189 (Approx. 53% of the graph)

- **Convergence:** Achieved in 84 iterations.

The high percentage of dangling nodes highlights the importance of the dangling vector correction (h) in the Power Method. Without explicit handling, over 50% of the probability mass would be lost during iterations. The results identify the university's homepage and admissions pages as the most central nodes.

Rank	Score	ID	URL (Truncated)
1	0.01988	2	http://www.hollins.edu/
2	0.00929	37	.../admissions/visit/visit.htm
3	0.00861	38	.../about/about_tour.htm
4	0.00807	61	.../htdig/index.html
5	0.00803	52	.../admissions/.../info-request.cfm
6	0.00716	43	.../admissions/apply/apply.htm
7	0.00658	425	.../library/resources/web_linx.htm
8	0.00599	27	.../admissions/admissions.htm
9	0.00557	28	.../academics/academics.htm
10	0.00445	4023	.../Sculpture/sld001.htm

Table 2.3: Top 10 PageRank results for the Hollins University dataset.